

|| DSA PRACTICE QUESTION ||

Here are some practice questions covering each DSA topic you mentioned..

Conditional Statements

1. Check Even or Odd: Write a program that checks if a given number is even or odd.
2. Largest of Three Numbers: Create a program that finds the largest of three numbers.
3. Grade Calculator: Write a program that takes a score and assigns a grade (A, B, C, etc.) based on the range it falls in.
4. Leap Year Check: Create a program to determine if a given year is a leap year.
5. Positive, Negative, or Zero: Write a program that checks if a number is positive, negative, or zero.
6. Divisibility Check: Write a program that checks if a number is divisible by both 5 and 11.
7. Vowel or Consonant: Create a program that checks if a given letter is a vowel or a consonant.
8. Number in Range: Write a function that checks if a number is within a specified range.
9. Triangle Validity: Write a program that checks if three given angles can form a triangle.
10. Day of the Week: Create a program that takes a number (1-7) and prints the corresponding day of the week.

Functions

1. Factorial Calculation: Write a function `factorial(n)` that returns the factorial of a given number `n`.

2. Prime Check: Create a function `isPrime(n)` that checks if a number `n` is prime.
3. Fibonacci Sequence: Write a function `fibonacci(n)` that returns the `n`-th Fibonacci number.
4. Palindrome Check: Implement a function `isPalindrome(s)` that checks if a given string `s` is a palindrome.
5. Greatest Common Divisor (GCD): Write a function `gcd(a, b)` that finds the GCD of two numbers `a` and `b` using recursion.
6. Count Vowels and Consonants: Create a function `countVowelsConsonants(s)` that returns the count of vowels and consonants in a given string `s`.
7. Sorting an Array: Write a function `sortArray(arr)` that takes an array `arr` and returns it sorted.
8. Power Calculation: Create a function `power(base, exponent)` that calculates the power of a base to an exponent.
9. Array Sum: Implement a function `arraySum(arr)` that returns the sum of all elements in an array `arr`.
10. Reverse String: Write a function `reverseString(s)` that reverses a given string `s`.

Loops

1. Print the first `n` Fibonacci numbers using loops.
2. Write a program to check if a number is prime.
3. Create a pattern with stars, like a pyramid.
4. Print all prime numbers within a given range.

5. Find the sum of digits of a number.
6. Calculate the factorial of a number.
7. Print a multiplication table for a given number.
8. Reverse a given number using loops.
9. Find the GCD of two numbers using a loop.
10. Check if a number is a palindrome.

Pattern

1. Print a right-angled triangle pattern using stars.
2. Create a diamond pattern of numbers.
3. Print an inverted triangle pattern.
4. Make a hollow square pattern.
5. Print Pascal's triangle for a given number of rows.
6. Create a pattern that displays numbers in ascending order in each row.
7. Print a zig-zag pattern using stars.
8. Make a butterfly pattern.
9. Print a Floyd's triangle.

10. Print a pattern that forms an hourglass shape.

Array

1. Find the maximum and minimum elements in an array.

2. Write a program to reverse an array.

3. Merge two sorted arrays.

4. Move all zeros to the end of an array.

5. Find the frequency of each element in an array.

6. Sort an array of 0s, 1s, and 2s without using sorting algorithms.

7. Find the second largest element in an array.

8. Write a program to rotate an array.

9. Find pairs in an array with a given sum.

10. Find the missing number in a sequence of n natural numbers.

String

1. Reverse a string without using extra space.

2. Check if two strings are anagrams.

3. Find the longest common prefix among an array of strings.

4. Count vowels and consonants in a string.
5. Implement a function to check if a string is a palindrome.
6. Find the first non-repeating character in a string.
7. Replace all spaces in a string with "%20".
8. Check if two strings are rotations of each other.
9. Find all permutations of a string.
10. Write a function to check if a string contains only digits.

Sorting

1. Implement Bubble Sort and analyze its complexity.
2. Write a program for Selection Sort.
3. Implement Insertion Sort and count the number of swaps.
4. Perform Merge Sort on an array.
5. Implement Quick Sort and calculate the number of comparisons.
6. Sort an array using Heap Sort.
7. Write a program to sort strings lexicographically.
8. Implement Bucket Sort for a given set of floating-point numbers.

9. Perform Radix Sort on an array of integers.
10. Count the number of inversions in an array using sorting.

Bit Manipulation

1. Find the only non-repeating element in an array where every other element repeats.
2. Swap two numbers without using a third variable.
3. Find the XOR of all elements in an array.
4. Count the number of set bits in an integer.
5. Determine if a number is a power of two.
6. Write a program to toggle case of all characters in a string.
7. Find two non-repeating elements in an array.
8. Write a function to check if the i-th bit is set in an integer.
9. Find the missing number in an array of consecutive numbers using XOR.
10. Multiply a number by 7 using bitwise operations.

Object-Oriented Programming (OOP)

1. Implement a BankAccount class with methods for deposit and withdrawal.

2. Create a Student class to manage details like name, age, and grades.
3. Design a Shape class with polymorphic area() methods for different shapes.
4. Implement a Library class to manage book records and lending services.
5. Create an Employee class with methods to calculate bonuses based on position.
6. Implement an inheritance hierarchy for Animal class with subclasses like Dog, Cat, etc.
7. Design a Vehicle class with properties and methods, and create child classes for different types.
8. Write a program demonstrating the concept of interfaces.
9. Create a Game class to manage player scores and rankings.
10. Develop a Calculator class with methods for basic operations and overload them.

Divide and Conquer

1. Implement a binary search algorithm.
2. Write a function to find the peak element in an array.
3. Solve the "Maximum Subarray Sum" problem using Divide and Conquer.
4. Implement a recursive version of Merge Sort.
5. Write a function to find the closest pair of points in a 2D plane.
6. Implement Quick Sort and analyze its complexity.

7. Write a program to count inversions in an array using Merge Sort.
8. Solve the "Power of a Number" problem using Divide and Conquer.
9. Find the median of two sorted arrays.
10. Implement the "Matrix Multiplication" problem using Divide and Conquer.